

第5讲：模型是怎么“学习”的？

Loss、梯度、反向传播与梯度下降

这一讲只解决一个核心问题

模型做错以后，它究竟怎样知道“哪些参数应该改、往哪个方向改、改多少”？理解这一点，你才真正知道“训练模型”是什么意思。

学习路线：预测 → 损失函数 → 导数/梯度 → 链式法则 → 反向传播 → 梯度下降 → 优化器 → LLM 下一词训练

零基础原则：不要求微积分推导；公式只保留理解 LLM 必须知道的部分。

01 本讲目标：把“训练”从黑箱变成可解释过程

- 理解学习的本质：不是把答案“存进去”，而是用数据反复调整参数。
- 理解 Loss：它把“错得有多严重”压缩成一个可优化的数值。
- 理解梯度：它回答“某个参数变化一点，Loss 会怎样变化”。
- 理解反向传播：它高效计算所有参数各自对 Loss 的影响。
- 理解梯度下降：它使用梯度真正更新参数。
- 连接 LLM：理解 next-token prediction 如何变成数十亿参数的学习信号。



最小闭环

一次训练 step 可以粗略写成：前向计算 → 计算 Loss → 反向传播得到梯度 → 优化器更新参数。以后看到 PyTorch 里的 `forward / loss.backward() / optimizer.step()`，就是这四步。

02 回顾上一讲：参数只是“可调旋钮”

上一讲我们把神经网络写成一个极简公式：

前向计算

$$h = f(Wx + b)$$

x 是输入， W 和 b 是参数， f 是激活函数。训练开始前， W 和 b 通常并不是“已经正确”的；训练的任务就是逐渐找到更合适的数值。

可以把每个参数想象成控制模型行为的一个微小旋钮。现代 LLM 有几十亿、几百亿甚至更多旋钮。难点不在于“能不能调”，而在于：一次预测错了以后，怎么知道这么多旋钮分别该怎么调？

概念	零基础直觉	训练中的作用
Parameter 参数	可以被调整的数值旋钮	被优化器修改
Forward 前向	拿当前参数算一次预测	产生预测值/概率
Loss 损失	把错误变成一个数字	告诉我们当前有多差
Gradient 梯度	Loss 对每个参数的局部敏感度	告诉更新方向与相对强度

常见误区

“训练”不是程序员手工决定每个参数该是多少，也不是每看到一条数据就写一条规则。真正的训练是一个自动数值优化过程。

03 模型怎样知道自己“错了”？——先定义目标

训练必须先有一个“我们希望模型做什么”的目标。否则模型根本不知道什么算好、什么算坏。

例子：房价预测。输入房屋面积 $x=100$ ，真实价格 $y=200$ 万；模型当前预测 $\hat{y}=150$ 万。我们直觉上知道它少预测了 50 万，但优化算法需要一个明确的数值规则。

误差 $\neq$ 损失函数

“预测差了 50 万”只是误差的一种描述。训练通常会把误差进一步变成 Loss。例如平方误差： $L=(y-\hat{y})^2$ 。

真实 y	预测 $\hat{y}$	误差 $y-\hat{y}$	平方损失 $(y-\hat{y})^2$
200	150	-50	2500
200	190	-10	100
200	199	-1	1

可以看到：预测越接近真实值，Loss 越小。于是“让模型变好”就可以转换成一个数学问题：想办法让 Loss 下降。

训练最核心的抽象

复杂任务最后常常会被转换成：寻找一组参数 θ ，使平均 Loss 尽可能小。 θ (theta) 只是“所有参数的集合”的常用记号。

04 Loss Function: 为什么一定要有“评分尺”？

Loss function（损失函数）相当于训练系统中的评分尺。没有评分尺，优化器就没有可追逐的目标。不同任务会设计不同的 Loss。

任务	模型输出	常见 Loss	直觉
回归	一个连续数值	MSE / MAE	预测值离真实值多远
分类	各类别概率	Cross Entropy	正确类别得到的概率够不够高
语言模型	下一个 Token 的概率分布	Cross Entropy	正确 Token 的概率是否提高

对 LLM 来说，最关键的是第三行：模型并不是直接输出“答案对/错”，而是对词表里的大量 Token 给出一个概率分布。训练数据告诉它“真正的下一个 Token 是哪个”，然后 Cross Entropy 对这个概率分布进行评分。

先只记直觉

如果正确 Token 的预测概率很高，Loss 较小；如果正确 Token 的概率很低，Loss 较大。具体公式后面会给，但这一讲不要求推导。

05 从“Loss 很大”到“该怎么改参数” 还差一步

仅知道 $\text{Loss}=2500$ 没有告诉我们参数该怎么改。就像你知道考试考了 50 分，但不知道哪道题错了、应该改哪部分。

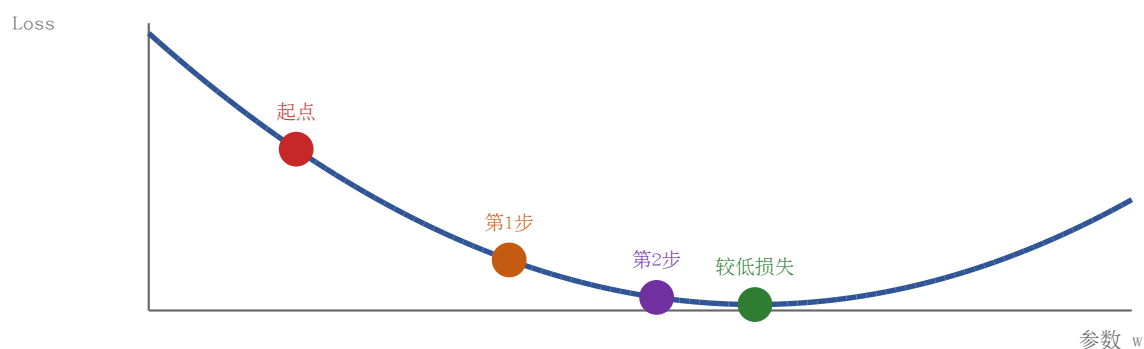
假设模型极简到只有一个参数 w ：

极简模型

$$\hat{y} = w \cdot x$$

如果 $x=2$ ，真实 $y=6$ ，那么理想情况下 $w=3$ 。假设初始 $w=1$ ，则预测 $\hat{y}=2$ ，显然偏低。

我们可以尝试很多 w ，算出每一个 w 对应的 Loss 。于是参数 w 和 Loss 之间形成一条“地形曲线”。训练就是在这张地形图上找低处。



梯度告诉我们局部坡度；梯度下降沿“下坡方向”移动参数

问题变成

站在当前参数位置，我们不需要一次看到整张地图；只需要知道脚下的坡度：往左会变高还是变低？往右会变高还是变低？坡有多陡？这就是导数/梯度提供的信息。

06 导数：先不要想微积分，先想“局部坡度”

导数 (derivative) 最有用的直觉是：输入发生一个很小变化时，输出大约变化多少。

导数符号	含义	对训练的启示
$dL/dw > 0$	w 增大一点，Loss 会增大	想降 Loss, w 应该往小方向改
$dL/dw < 0$	w 增大一点，Loss 会减小	想降 Loss, w 应该往大方向改
$dL/dw \approx 0$	附近比较平	可能接近低点，也可能进入平坦区

例如 $L=(w-3)^2$ 。当 $w=1$ 时，导数是 $2(w-3)=-4$ 。负号告诉我们：把 w 增大，Loss 会下降。

注意

“导数为负”不是说 Loss

本身为负，而是说曲线在当前位置向右走时正在下降。导数描述的是变化趋势。

当只有一个参数时叫导数；当参数很多时，会同时得到对每个参数的偏导数，它们放在一起就是梯度 (gradient)。

07 Gradient: 几十亿参数时，就是几十亿个“局部坡度”

假设模型只有两个参数 w 和 b ，Loss 写成 $L(w,b)$ 。一次反向传播之后，我们会得到：

两个梯度分量

L/w w : w 改一点对 Loss 有什么影响

L/b b : b 改一点对 Loss 有什么影响

如果模型有 100 亿个可训练参数，那么梯度也可以理解成一个包含约 100 亿个分量的巨大结构——每个参数都有自己的梯度。

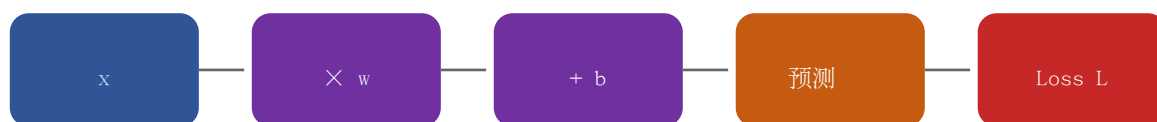
参数	当前值	梯度（示意）	解释
w	0.72	+0.031	增大 w 会让 Loss 增大一点
w	-1.10	-0.004	增大 w 会让 Loss 略微下降
b	0.08	+0.120	Loss 对 b 相对敏感

重要：梯度不是“正确答案”

梯度只描述当前参数位置附近的局部方向。参数更新以后，模型到了新位置，需要重新做前向与反向传播，再算新的梯度。

08 为什么需要链式法则？——一个参数藏在很多层后面

真实神经网络不是一个简单公式，而是一层接一层：参数先影响中间表示，中间表示再影响后面的层，最后才影响 Loss。



前向：从左到右算结果；反向：从右到左传播“每个中间量对 Loss 的影响”

链式法则把局部影响一段一段乘起来，于是能算出 L/w 、 L/b

假设 w 先影响 z ， z 再影响预测，再影响 Loss。我们真正想知道的是“ w 对 Loss 的总影响”。链式法则告诉我们可以把沿途的局部影响连接起来。

只记结构，不要求推导

$$L/w = (L/\text{预测}) \times (z/\text{预测}) \times (z/w)$$

可以把它理解成：每一段都告诉你“上游变化 1 点，下游会变化多少”，然后把整条路径的影响串起来。

链式法则本身是数学规则；反向传播（backpropagation）是把这条规则组织成一个高效算法，避免对每个参数从头重复算一遍。

09 Backpropagation: 反向传播到底“传播”什么？

反向传播不是把“答案”传回去，也不是把训练数据倒着走一遍。它传播的是：Loss 对各个中间变量的梯度信息。

阶段	方向	主要做什么
Forward 前向传播	输入 → 输出	保存必要的中间结果，得到预测与 Loss
Backward 反向传播	Loss → 前面各层	用链式法则计算每个参数的梯度
Update 参数更新	优化器执行	根据梯度调整参数

为什么叫“反向”？因为计算梯度时通常从最终 Loss 开始，沿计算图向前面的运算节点反向推进。

核心区别

Backprop 负责算梯度；Gradient Descent / Adam 负责使用梯度改参数。
这是初学者最容易混为一谈的两个步骤。

现代深度学习框架（如 PyTorch）会自动记录前向计算组成的 computation graph（计算图），然后自动求导。这个机制通常叫 automatic differentiation / autograd。

10 Gradient Descent: 真正修改参数的规则

有了梯度，就可以更新参数。最基本的梯度下降规则是：

参数更新公式

$$\theta_{\text{new}} = \theta_{\text{old}} - \eta \cdot L$$

θ 表示参数； L 表示梯度； η (eta) 是学习率 learning rate。

为什么前面有减号？因为梯度指向“Loss 增长最快”的局部方向；我们希望 Loss 下降，所以反方向走。

数值例子： $w=1$ ，梯度 $dL/dw=-4$ ，学习率 $\eta=0.1$ ：

一步更新

$$w_{\text{new}} = 1 - 0.1 \times (-4) = 1.4$$

因为梯度为负，更新后 w 变大。对前面的例子而言，这正是在朝理想值 $w=3$ 靠近。

然后用 $w=1.4$ 再做一次前向计算，得到新的

Loss，再算新的梯度，再更新。训练就是这样反复进行很多很多次。

11 Learning Rate: 为什么“走多大一步”非常关键



学习率决定每次沿梯度方向更新多远。它不是模型自动“知道”的参数，而是训练时非常重要的超参数（hyperparameter）。

情况	现象	可能结果
学习率太小	每一步参数变化很小	训练非常慢，成本高
学习率合适	Loss 较平稳下降	较高效收敛
学习率太大	一步跨过低谷，来回跳	Loss 震荡或发散

现实训练更复杂

大模型训练常用学习率预热（warmup）、衰减（decay）等策略，而不是从头到尾使用一个固定值。但底层直觉仍然是：控制每一步更新的尺度。

12 Batch、Step、Epoch：训练日志里的三个高频词

真实数据太多，不可能每更新一次参数就把全部数据都算一遍。因此通常把训练数据分成一小批一小批。

术语	含义	例子（1000 条数据，batch=100）
Batch	一次拿来计算梯度的一批样本	每批 100 条
Step / Iteration	完成一次参数更新	处理一个 batch 后更新一次
Epoch	所有训练样本大致被看过一遍	10 个 batch $\approx$ 1 epoch

LLM 的数据规模通常巨大，训练论文经常不用“epoch”作为主要进度单位，而会报告 training tokens、steps、compute 等，因为语料可能只过一遍甚至不足一遍。

为什么 mini-batch 可行？

一个 batch 给出的梯度只是“全体数据真实平均梯度”的近似，但计算成本大幅降低，而且这种噪声有时反而有助于优化。

13 SGD、Momentum、Adam： 优化器在做什么？

最原始的 SGD (Stochastic Gradient Descent) 直接按当前 mini-batch 梯度更新。现代大模型训练通常会使用更复杂的优化器，例如 Adam / AdamW。

优化器直觉	核心理念	你现在需要记什么
SGD	按梯度直接走	最基本的优化规则
Momentum	参考过去一段时间的方向	像小球带惯性，减少来回抖动
Adam	为不同参数自适应调整更新尺度	深度学习中非常常见
AdamW	把 weight decay 处理得更合理	LLM 训练常见选择之一

不要把优化器想成 “智能算法”

优化器不会理解文字、知识或任务含义。它只接收数值梯度，并按照既定数值规则更新参数。模型能力来自数据、架构、目标和大规模优化共同作用。

14 LLM 的训练目标：把“下一个 Token”变成监督信号

现在把前面的知识接回语言模型。给定序列：

训练样本（示意）

输入上下文：“中国的首都是”

真实下一个 Token：“北京”

模型输出：对整个词表每个 Token 给一个概率。

假设模型当前输出：

候选 Token	模型概率
北京	0.10
上海	0.25
中国	0.05
纽约	0.03
其他大量 Token	合计 0.57

训练数据告诉我们正确 Token 是“北京”。Cross Entropy 会对“北京只有 0.10 概率”给出较大的惩罚。反向传播随后会计算：哪些参数应该如何改变，才能让类似上下文中“北京”的概率更高。

关键认识

一次训练不是“把北京=中国首都写进某个地址”，而是一个极其分布式的参数更新过程；许多权重会发生很小变化。

15 Cross Entropy: 为什么正确 Token 概率越低，惩罚越大？

对单个正确 Token，常见的负对数似然形式可以写成：

单位置 Loss（简化）		
$L = -\log p(\text{correct token})$		
正确 Token 概率 p	$-\log(p)$ （约）	含义
0.90	0.105	模型很有把握，Loss 小
0.50	0.693	不确定，Loss 中等
0.10	2.303	正确答案概率很低，Loss 大
0.01	4.605	几乎没押中，惩罚更大

一段文本会有很多 Token 位置，每个位置都可以产生一个 next-token prediction Loss，最后通常会做求和或平均。于是海量文本自动提供了海量监督信号。

为什么“自监督”？
不需要人工给每句话贴标签。文本本身就提供答案：前面的 Token 是输入，真实出现的下一个 Token 就是目标。

16 一次 LLM 训练 step，内部究竟发生什么？

顺序	步骤	发生的事情
1	取一个 batch 的 Token	得到输入序列与目标 Token
2	Forward	经过 Embedding + 多层 Transformer，得到 logits
3	Softmax + Loss	把 logits 变成概率，计算 Cross Entropy
4	Backward	Autograd 计算所有可训练参数的梯度
5	Optimizer step	AdamW 等优化器更新参数
6	清理/继续	清梯度，进入下一 batch

这套闭环和你在这讲开头看到的极简房价模型，在逻辑上是同一件事。差别只是：LLM 的计算图巨大得多，参数更多，数据更多，硬件与系统工程复杂得多。

从“小公式”到“大模型”

规模变了，但最底层逻辑没有变：前向得到预测 → Loss 评价 → 反向算梯度 → 优化器更新参数。

17 为什么训练很贵，而推理相对便宜？

推理时通常只需要 forward；训练除了 forward，还必须保存大量中间激活，并进行 backward，还要保存梯度以及优化器状态。

阶段	需要什么	额外内存/计算
Inference 推理	主要做前向计算	相对较少
Training 训练	前向 + Loss + 反向 + 参数更新	计算和显存需求显著增加

例如 Adam

类优化器常常需要为每个参数维护额外状态。再加上激活值、梯度、参数本身，一个“70B 参数模型”训练时的显存需求远不能简单按“参数文件大小”来估算。

这与 AI Systems 有什么关系？

大模型训练不是纯算法问题，还涉及并行计算、显存管理、通信、检查点、容错、吞吐等系统问题。这也是“AI Systems”成为重要研究方向的原因之一。

18 为什么 GPU 对训练如此重要？

梯度计算本质上仍然由大量矩阵乘法、加法和逐元素运算组成。GPU 可以用大量并行计算单元同时处理这些数值操作。

CPU	GPU	训练意义
核心较少但单核能力强、控制灵活	大量并行计算单元	矩阵运算可大规模并行
擅长复杂流程与通用任务	擅长高吞吐数值计算	Transformer 前向和反向都受益

不过“GPU 多”并不自动等于训练快。多卡训练还要处理数据并行、张量并行、流水线并行以及卡间通信。规模越大，系统效率越重要。

与个人研究者的关系

你研究 Agent Systems 并不需要自己从零预训练基础模型。商业 API 或开源模型推理通常足够支撑大量机制型 Agent 实验；理解训练原理主要是为了正确理解模型能力与限制。

19 最常见的 8 个误区

- 误区1: Loss 下降就等于模型已经“理解”世界。——Loss 是训练目标上的数值指标，不等同于哲学意义上的理解。
- 误区2: 反向传播负责修改参数。——它主要负责计算梯度；优化器才执行更新。
- 误区3: 梯度就是误差。——梯度是误差/Loss 对参数变化的敏感度。
- 误区4: 梯度下降一次就找到最佳参数。——通常需要大量 step。
- 误区5: 梯度越大越好。——过大可能导致不稳定，需要裁剪、归一化等技术。
- 误区6: Loss 越小，真实世界表现一定越好。——还要看泛化、数据分布和评测。
- 误区7: 参数里的每个数字对应一条知识。——知识通常分布在大量参数和表示中。
- 误区8: LLM 只是在“背训练数据”。——记忆确实存在，但训练还会学习统计结构、模式和可泛化表示；具体机制仍是活跃研究问题。

科研阅读提示

论文里看到 `training objective`、`loss landscape`、`optimizer`、`learning rate`、`gradient clipping`、`warmup`、`backprop`、`autograd` 时，你现在已经有了最基本的概念框架。

20 小实验 A：亲手看一个参数怎样一步步学到正确值

材料包中的 `04_gradient_descent_demo.py` 不依赖第三方库。它训练极简模型 $\hat{y} = w \cdot x$ ，让 w 从随机/错误值逐步靠近目标。

```
# 初始化参数
x = 2.0
y = 6.0
w = 0.5
lr = 0.1

for step in range(10):
    y_hat = w * x
    loss = (y_hat - y) ** 2
    grad = 2 * (y_hat - y) * x
    w = w - lr * grad
    print(step, w, loss, grad)
```

你要观察三件事：① Loss 是否整体下降；② 梯度正负如何改变更新方向；③ w 是否逐渐靠近 3。

实验不是为了编程

这段代码最重要的是把训练过程“去神秘化”：只有 预测、算 Loss、算梯度、更新参数 四件事。现代 LLM 只是把这四件事放大到了极端规模。

21 小实验 B: 用数值梯度验证“梯度到底在说什么”

同一个脚本还会用 finite difference（有限差分）近似梯度：把参数 w 稍微增加 ϵ ，再观察 Loss 改变多少。

数值梯度思想

$$dL/dw \approx [L(w+\epsilon) - L(w-\epsilon)] / (2\epsilon)$$

它不是深度学习训练真正使用的高效方法，但非常适合初学者验证自动/解析梯度是否合理。

如果解析梯度和数值梯度非常接近，就说明你的梯度公式大概率没有写错。这种思想也叫 gradient checking。

材料包中的脚本会同时打印解析梯度和数值梯度，方便你直接比较。

进一步理解

Backprop 的价值不是“只有它才能求导”，而是：当网络有海量参数和层级时，它可以复用中间结果，高效地一次计算出所有参数的梯度。逐参数做有限差分会贵得不可接受。

22 全讲总结：把训练闭环压缩成一张心智地图



重复很多次：参数逐步变得更合适

你看到的词	最简含义
Loss	当前预测 “有多差” 的数值评分
Derivative / Gradient	Loss 对参数局部变化的敏感度
Chain Rule	把多层计算中的局部影响串起来
Backpropagation	高效计算大量参数梯度的算法
Gradient Descent	沿梯度反方向调整参数的基本思想
Optimizer	更实际地利用梯度更新参数的规则
Learning Rate	每一步参数更新的尺度

你现在应该能回答

“模型为什么会学习？”——因为训练数据定义目标，Loss 把错误量化，反向传播计算每个参数对 Loss 的影响，优化器再按梯度不断调整参数。重复海量次以后，参数逐渐形成能完成任务的计算结构。

下一讲：训练集、验证集、测试集、过拟合与泛化——为什么训练 Loss 很低，模型仍然可能在真实世界失败？